Privacy Preserving Continuous Big Data Publishing

Salheddine Kabou
Higher Normal School of Bechar
Bechar, Algeria
kabou.salheddine@ensbechar.dz

Laid Gasmi
Ahmed Draya University -Adrar
Adrar, Algeria
lai.gasmi@univ-adrar.edu.dz

Abdelbaset Kabou
Scientific and Technical Information Center
Algiers, Algeria
kabou.abdelbasset@gmail.com

2024 4th International Conference on Embedded & Distributed Systems (EDiS) | 979-8-3315-1987-2/24/$31.00 ©2024 IEEE | DOI: 10.1109/EDIS63605.2024.10783344

***Abstract*—The main challenge in big data analytics is preserving individual privacy. Anonymization models like k-anonymity and l-diversity are used to balance data privacy and utility during publishing. However, these models focus on a single data release and ensure a specific level of privacy. In practical big data applications, data publishing is more complex as it occurs continuously with new data collection, necessitating privacy for different releases. In this research, which is the primary research that focuses on the publication of continuous big data, a new distributed bottom-up approach is introduced to achieve the m-invariance privacy model in continuous big data scenarios. This approach is based on data insertion and splitting. Data records from various workers are inserted into an improved ascending R-tree generalization to minimize information loss. The second phase involves splitting overloaded nodes by reducing overlaps among resulting partitions according to the m-invariance model. Experimental results demonstrate significant improvements in data utility and execution time, compared to existing approaches in literature.**

***Index Terms*—Anonymization, M-invariance, Apache Spark**

I. Introduction

In recent years, many institutions need to share large data sets for research purposes. A key issue is how to protect individual privacy while sharing this data. This challenge led to the development of Privacy Preserving Data Publishing (PPDP), a field aiming to find a balance between data privacy and utility [1]. With growing data and the advent of big data paradigm, various privacy models like k-anonymity [2] and l-diversity [3] have been widely used, known as Privacy-Preserving Static Data Publishing (PPSDP). These models focus on single dataset publication and ensure a certain level of privacy protection.

In the realm of Big Data, where data is characterized by the 42 Vs, data publishing becomes more intricate [4]. In this context, organizations publish data in real time, incrementally sharing newly collected data and performing anonymization for different purposes. Dynamic data publishing poses a challenge for traditional static privacy models, which focus on single releases. This leads to the need for ensuring big data privacy while adapting to dynamic data publishing [5]. Privacy models like m-invariance address this and offer more suitable privacy levels than static models [6].

To anonymize data effectively, the generalization method is crucial. This involves replacing old values of quasi identifier (QID) attributes with more general values based on a given taxonomy tree, ensuring semantic consistency with previous data. Two main approaches to navigate the taxonomy tree for generalization exist. The first is Top–Down Specialization (TDS), changing values from the top downwards. The second, Bottom–Up Generalization (BUG), involves generalizing data from the tree's bottom to its top [7]. In our approach, we use the BUG method for better generalization efficiency compared to top-down [8].

Dealing with vast data volumes like zettabytes requires efficient processing time for sharing and publishing. Traditional centralized techniques fall short in time complexity, scalability, and performance. Distributed frameworks like MapReduce and Apache Spark have arisen to handle big data. Spark, an open-source platform, excels in scalable processing. Programs in this platform run much faster, making it a superior choice compared to MapReduce [9]. Prior studies have predominantly employed static privacy models with MapReduce or Spark platforms. However, as far as we know, no prior study has applied a privacy-preserving continuous data publishing model on the Apache Spark framework. Our current work aims to address this gap by implementing a distributed bottom-up m-invariance model using Apache Spark. Our objective is to enhance privacy, data utility, and processing speed.

Main contributions of this paper are summarized as follows:

- Introducing a new multidimensional bottom-up anonymization method for continuous big data publishing. It's implemented on Apache Spark using RDD programming, adhering to m-invariance privacy requirements.
- Presenting a new bottom-up algorithm that enhances R-tree generalization by grouping data records from multiple executors. The insertion and split process minimizes information loss compared to top-down Mondrian algorithms.

The rest of this paper is structured as follows:. - Section 2 explores related works in the literature. Section 3 details the fundamental concepts and primary stages of our distributed privacy model. Section 4 focuses on evaluation results and discussions. Finally, section 5 concludes and highlights potential future directions.

II. Related work

This section introduces distributed big data algorithms for anonymization, employing either top-down or bottom-up generalization operators.

Fung et al [10] presented a top-down specialization (TDS) anonymization algorithm based on the k-anonymity model

979-8-3315-1987-2/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on June 23,2025 at 11:18:52 UTC from IEEE Xplore. Restrictions apply.

for Apache Spark platform. It handles both numerical and categorical attributes and employs an iterative process to to realize the specialization process from the most general node. Zakerzadeh et al. [11] adapted the centralized top-down Mondrian algorithm to big data using MapReduce. Their aim was to improve processing efficiency across computational nodes, like Mapper, Combiner, and Reducer, in the context of large datasets. Ashkouti et al. [12] have extended prior research by implementing the multi-dimensional top-down Mondrian's algorithm within the Apache Spark ecosystem. The distinguishing feature between these two algorithms lies in the axis dimension selection criterion for the split operation. In [11], the algorithm opts for the attribute with the widest domain as the axis dimension. However, in this enhanced study, the criterion involves selecting the axis dimension that best disperses attribute values across the domain. The aim is to accelerate data anonymization through RDD programming and elevate anonymity levels using the l-diversity model, instead of k-anonymity.

The centralized traditional BUG methods are unsuitable for fulfilling privacy requirements in the context of big data. This limitation prompted researchers to propose parallel bottom-up methods to overcome the constraints associated with centralized BUG techniques.. In [13] and [14], an advanced scalable approach for k-anonymity using MapReduce on Cloud is introduced. Original data is split into tables, parallel anonymization generates intermediate results. These results are grouped, followed by an additional anonymization step to ensure privacy. The parallel method is shown to be more efficient and scalable than top-down specialization.

Our research differs from the mentioned studies by focusing on the continuous data publishing scenario within the big data paradigm. Notably, the most of prior techniques primarily targeted privacy preservation on the MapReduce platform, whereas our work introduces a bottom-up distributed algorithm designed for the Apache Spark ecosystem. This algorithm is tailored to tackle challenges posed by continuous big data publishing using the m-invariance model instead of the l-diversity, and minimizing the execution time and information loss

III. Proposed method

A. Foundational principles

Privacy-preserving dynamic data publishing (PPDDP) is a crucial research that encompasses three primary scenarios: Multi-dimensional Publishing: In this scenario, the original data table is simultaneously published across multiple dimensions. Sequential Release Publishing: The data is released to various recipients over time as new information becomes available. This is done vertically, meaning all releases share the same records but differ in attributes. Continuous Data Publishing: This scenario involves a fixed set of attributes but dynamically updated records through insertions, updates, or deletions, maintaining a horizontal distribution.

In this context, the data is structured in a table format, containing various attributes. These attributes are categorized as follows: (i) Explicit Identifiers (EI): Attributes like "Name" that directly identify an individual. (ii) Quasi Identifiers (QI): Attributes such as "Age" that, when combined with external information, can lead to individual identification. (iii) Sensitive Identifier (SI): This attribute holds sensitive values. Notably, attributes EI and QI can be either numerical or categorical, while SI is categorical in nature.

To ensure the data anonymization of such table, it is necessary to convert this table into a set of generalized disjoint groups, denoted as Equivalence Class (EC) or QI-group. The l-diversity model requires each EC contains at least l- distinct sensitive values. In the continuous data publishing, if a sensitive value was removed in a prior version, its absence in the present one can result in a situation where privacy is compromised. In order to overcome this problem, the idea of the m-invariance model [11] is to create new fake records (counterfeit records) in place of missing sensitive values

Definition 1: A set of publications P1,.....Pn is said to be m-invariant if: (1) Every publication satisfies the l-diversity model (records in every EC have different sensitive values). (2) Every individual has the same label in every EC

Definition 2: The label of each data record is the set of sensitive values of the equivalence class to which it belongs.

B. The stages of the proposed method

Fig 1 illustrates the primary stages of our strategy aimed at realizing the distributed m-invariance model. A series of RDDs are executed through transformations and actions available in the Apache Spark library.

1) Data preparation: The data acquired from the data holder contains numerous instances of missing and duplicated records. The core aim of this stage is to construct a suitable version of the initial data that aligns with the requirements of the anonymization procedure. Through the removal of erroneous records, the resultant data is distributed across various worker nodes.

2) Initial generalization: During this stage, the workers simultaneously carry out data anonymization using the initial generalization process. The simple l-diversity model is achieved in this phase for producing intermediate results

3) Mapping operation: In this step, the mapping operation is executed to establish a connection between the $< key, value >$ pairs of each data record (Algorithm 1). As a result of the preceding phase, a collection of ECs is obtained, adhering to the l-diversity model. In this context, the key signifies the label of each data record, encompassing the set of distinct sensitive values within the EC in which it belongs. Meanwhile, the value corresponds to the attribute value within each data record.

4) Insertion process: Insertion can be accomplished through two distinct levels:

a. In the initial step, pairs with identical keys are grouped together, ensuring they reside on the same worker nodes.

b. The proposed method focuses on the insertion algorithm within a big data context, accommodating continuous and real-time data releases by a data publisher. The aim is to enhance

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on June 23,2025 at 11:18:52 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Mapping operation

Input: RDD2
Output: (key, value) pairs as RDD3

```
0: Key = [ ] // list of SI of each EC
0: Values = [ ]
0: for each data records in EC do
0:     Value = Value.append (QI)
0:     Key = All_SI (value)
0: end for=0
```

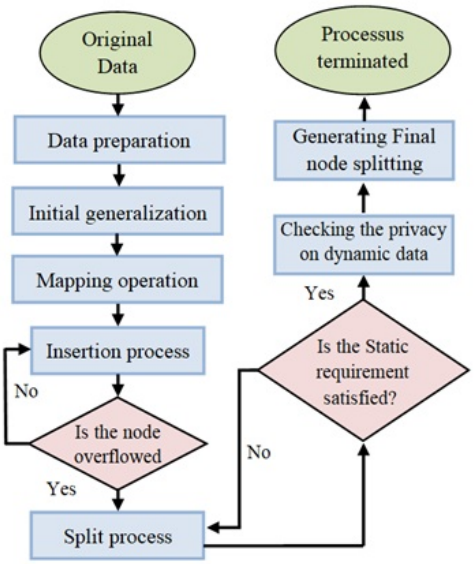

Fig. 1. Overview of the proposed method

the generalization of datasets by incorporating data records into ECs while upholding privacy requirements that prevent their division. Algorithm 2 employs an efficient bottom-up approach to recursively insert data records from different executors into the proper equivalence class (PEC). If the privacy requirement is breached, the EC domain undergoes partitioning along the optimal attribute dimension (refer to section "Split process"). Given that the reduction operation relies on data records sharing identical keys, the concept involves calculating the area difference between each data record and the available EC. This assessment helps in selecting the nearest data records. Opting for the minimum distance between the inserted data record and available equivalence classes is driven by the intention to minimize information loss during the generalization process. This ensures that the results are anonymized with minimal information loss.

5) Split process:

a) Optimal dimension selection:

Previous research has explored methods for choosing the optimal axis dimension in multidimensional algorithms. In the centralized Mondrian algorithm, the axis with the widest domain attribute is often chosen. However, this approach disregards the dispersion of attribute values across the domain. A more effective selection criterion is proposed in [12], which employs the Coefficient of Variation (CV) values. Another approach, detailed in [15], adopts the ChooseSplitAxis algorithm from the R*-tree method for bottom-up multidimensional anonymization. This algorithm minimizes overlap between distribution groups. However, it's deemed unsuitable for a Spark environment due to its potential impact on execution time. Considering these factors, the proposed method favors the use of the Coefficient of Variation (CV) as the criterion for selecting the axis dimension. Hence, the result of this stage is stored in RDD5, containing pairs of $< Key, Att >$. The key represents distinct Sensitive Identifiers within the equivalence class, while Att indicates the chosen attribute for splitting based on the optimal Coefficient of Variation value

Algorithm 2 Insertion process

Input: RDD3
Output: PEC

```
0: n = number of QI
0: for each pair of ECs having the same key do
0:     for (i = 1; i <= n; i++) do
0:         a = calculate Area Difference
0:         PEC = min (a)
0:     end for
0: end for=0
```

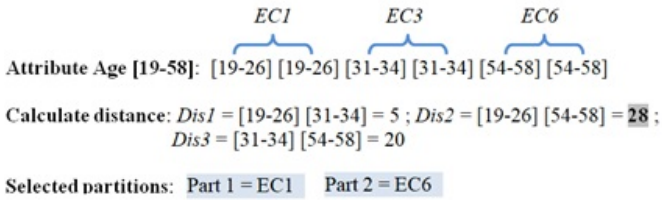

Fig. 2. Partitions Determination

b) Node splitting:

This phase involves node splitting on overflowed nodes. Prior work like [11] used a top-down Kd-tree approach, considering static data. Here, a bottom-up strategy is employed using an enhanced R-tree (Algorithm 3) that accommodates continuous data. The aim is to find a suitable partition for nodes with excess records while adhering to privacy requirements. The following steps are carried out to achieve the node splitting process.

Partition Determination (step1): The algorithm focuses on splitting overflowed leaf nodes, encompassing bounds of equivalence classes inserted from the initial l-diversity tables. The objective is twofold: not only dividing nodes but also minimizing overlap among resulting partitions, thereby boosting data utility. The approach involves isolating two equivalence class domains with the maximum distance between them. An illustration of this is shown in Fig 2. In the example, the attribute "age" has three QI-groups, and the aim is to select two equivalence classes with the greatest distance between them.

Partition filling (step2): As the initial group tends to be smaller, a point arrives when a new data record is prepared for addition to one of the two partitions. The choice is based

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on June 23,2025 at 11:18:52 UTC from IEEE Xplore. Restrictions apply.

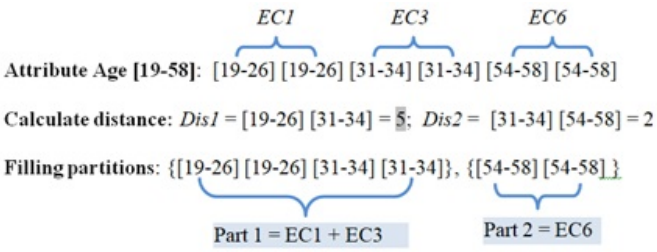

Fig. 3. Partitions Filling

on the partition with the minimum distance to the new entry in terms of equivalence class bounds. In Fig 3's example, the new equivalence class [31-34] is closer in distance to the partition containing [54-58] compared to the second partition. Consequently, the first partition expands its equivalence class domain to [31-58].

Privacy Requirement Check for Static Data (step3): In this phase, data is partitioned extensively while upholding the l-diversity model, aiming to minimize information loss. Key criteria include classifying all data records so that each partition holds a minimum of k elements and each equivalence class contains at least l distinct sensitive values. It's noteworthy that the "Partition Filling" process iterates until both partitions conform to the l-diversity model.

Privacy Requirement Check for Continuous Data (step4): This stage addresses the continuous data publishing challenge, aiming to maintain the distributed m-invariance model. In this scenario, data is realized multiple times, with each publication being an updated version of the previous one. The challenge lies not only in ensuring an anonymization model for each version but also in preventing attacks that compromise data privacy across all releases. This entails safeguarding against comparisons between different releases and the potential removal of sensitive values to avoid privacy breaches.

In the previous operations, the two partitions are selected, such that the requirement of the privacy static publishing is satisfied. After that, it is necessary to verify whether or not the location of SI relative to any EC achieves the conditions of privacy continuous data publishing. Given that each group adheres to the l-diversity model, the challenge revolves around achieving the m-invariance model across all executors. The process, as outlined in (Algorithm 3), encompasses three essential levels: The first level, the simplest, involves comparing the sets of sensitive identifiers from consecutive releases. If their labels match, individual privacy remains secure against adversarial threats. The second level aims to align sets of labels across releases while minimizing the insertion of counterfeit records. This level focuses on grouping other ECs with the same keys that didn't satisfy the first condition. To enhance data utility, selected ECs for grouping should maintain a minimum distance among other QI-groups. The third level addresses situations where the preceding conditions are unmet. Here, the approach employs the concept of counterfeited generalization from the m-invariance model to mitigate privacy risks.

Algorithm 3 Node splitting

Input: RDD5
Output: Part 1 and Part 2
0: **Step 1: The partitions determination**
0: **for** each pair of (EC_i, EC_j) along the best attribute $(i \neq j)$ **do**
0: Dis = area (EC_i) – $area(EC_j)$ // calculate the distance
0: **end for**
// Select the pair with the highest Dis
0: Part1 = EC_i
0: Part2 = EC_j
0: **Step 2: The partitions filling**
0: **for** each EC_i not inserted in one of the partitions **do**
0: // calculate the distance
0: Dis1 = area (Part1) – area (EC_i)
0: Dis2 = area (Part2) – area (EC_i)
0: **end for**
// Select the partition that results the minimum difference
0: Part j = Part j + EC_i // update
0: **Step 3: Checking the privacy on static data**
0: **repeat**
0: Step 2
0: **until**
0: The two partitions satisfy the condition of l-diversity
0: **Step 4: Checking the privacy on dynamic data**
0: **if** $P_{n-1}(key, originalValue) = P_n(key, originalValue)$ **then**
0: Generating final node splitting
0: **else if** $P_{n-1}(key, originalValue) = P_n(key, anyValue)$ **then**
0: Grouping the data record with EC having the minimum distance.
0: Generating final node splitting
0: **else**
0: Add the counterfeit records to EC
0: Generating final node splitting
0: **end if**=0

IV. Experimental evaluation

To validate our proposed algorithm, we conduct a comparative analysis in this section. We assess the performance of the proposed approach against established state-of-the-art techniques with a focus on data utility impact and execution time

A. Configuration

In our study, we employed the "Adults" dataset sourced from the UCI machine learning repository. This dataset comprises 48,842 data entries encompassing 14 attributes. Notably, we designated the attributes "Age," "Occupation," "Relationship," "Education," and "Workclass" as quasi-identifiers, while "Disease" was considered the sensitive identifier. The categorization structure for these quasi and sensitive identifiers aligns with the approach outlined in [16]. Given the context of big data, we found it necessary to augment the dataset's size. To achieve this, we expanded the "Adult" dataset through random processes that adhered to the selected quasi-identifiers. This expansion facilitated the creation of new data entries. The specifics of the enhanced "Adult" dataset are elaborated in table 1

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on June 23,2025 at 11:18:52 UTC from IEEE Xplore. Restrictions apply.

TABLE I
ADULT GENERATING TABLE

Dataset	Number of data records	Size
Adult 48k	48842	1.5 MB
Adult 500K	500 Thousand	15 MB
Adult 1M	1 Million	31 MB
Adult 10M	10 Million	295 MB

In static data publishing, one dataset view is assessed for information loss. In continuous big data release, multiple releases are used to validate the model. The initial publication, P1, has 500k records randomly selected. In subsequent releases (Pi), 10k records are removed from the previous version, and 15k new ones are added from the synthetic "Adult" dataset. This is repeated 20 times, resulting in 20 republished versions.

B. Information loss

Information loss, a crucial factor, quantifies the extent of information loss resulting from generalization processes. Various metrics can gauge this parameter, with the Normalized Cardinality Penalty (NCP) metric being the most suitable among the other measures measures [12], especially in the context of the big data paradigm.

$$NCP = \sum_{n=1}^{x} \sum_{m=1}^{y} \frac{|upper_{nm} - lower_{nm}|}{x.y|Max_m - Min_m|} \quad (1)$$

Equation (1) defines the NCP calculation formula. The numerator involves subtracting the upper and lower domains of attribute m in data record n after the generalization. The denominator signifies the range between the maximum and minimum attribute m values across the entire dataset. Here, x represents the data record count and y stands for the attribute count.

Mondrian, DIMondrian, and our approach were assessed using a synthetic "Adult" dataset across three aspects: initial publication, data records insertion and deletion. The evaluations were guided by the domains of k = 100 and l ranging from 3 to 6.

In fig 4, it's evident that NCP value escalates as k and l parameters increase for all three algorithms. This phenomenon arises due to the augmentation of data records within equivalence classes. The initial publication experiments demonstrated that both our and DI methods exhibit lower information loss compared to MapReduce-Mondrian. This stems from their superior selection criteria for node splitting. While the two first approaches incorporate all data values for axis dimension selection, Mondrian only considers upper and lower values, leading to less optimal decisions

In continuous data publication, Mondrian and DIMondrian were implemented by anonymizing new data separately and combine results with previous data. The approach employs values of k = 100 and l= 4 for 20 publications. Results (fig 5) show our method has less information loss than Mondrian methods due to our approach's focus on reducing overlap between Equivalence Classes and strategic selection of ECs during insertion.

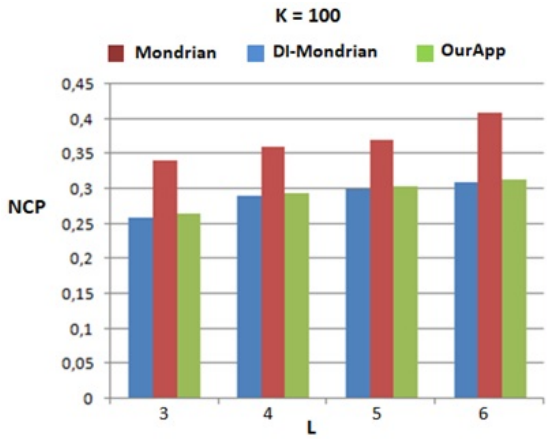

Fig. 4. Information loss for the first publication

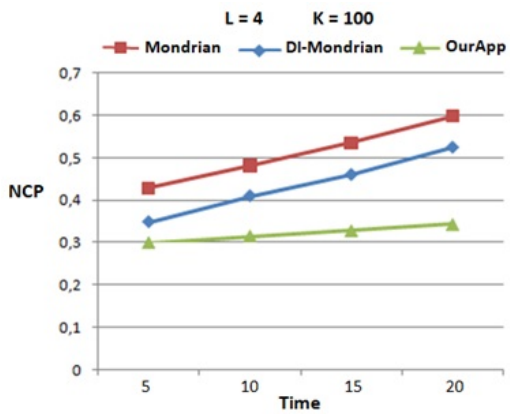

Fig. 5. Information loss for the continuous publication

C. Execution time

Apache Spark, a swift ecosystem for large data, uses in-memory storage to speed up processing by 100x compared to MapReduce in memory and 10x on disk. However, Comparing the execution time of MapReduce-Mondrian with our approach is methodologically challenging, even with identical configurations [12]. Fig 6, 7 and 8 display execution time of our method and DI-Mondrian on synthetic "Adult" data, measured in seconds. Anonymization configuration used l values from 3 to 6, with publication counts of 15. Tests included "Adult" datasets of sizes 500K, 1M, and 10M. It is observed that DI-Mondrian's execution time is slightly higher due to increased cut points and complex calculations. In our approach, the split process generates two groups in each iteration through a binary tree structure. In contrast, the DI Mondrian method might generate more partitions, especially as the r parameter increases. Consequently, a higher number of partitions in an overflowing node can occasionally lead to increased running times.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on June 23,2025 at 11:18:52 UTC from IEEE Xplore. Restrictions apply.

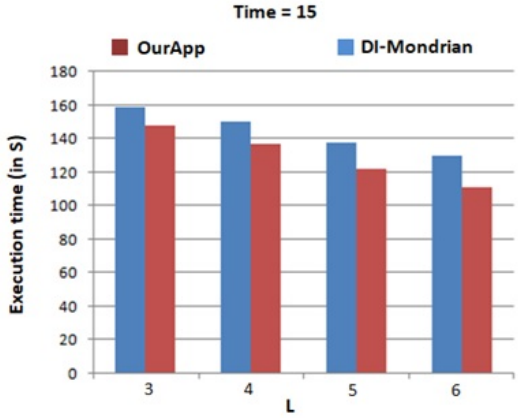

Fig. 6. Execution time for Adult 500K

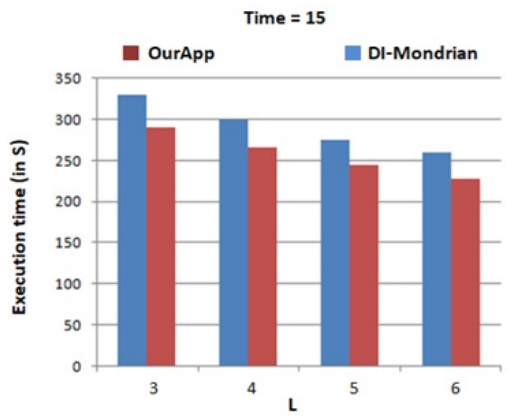

Fig. 7. Execution time for Adult 1M

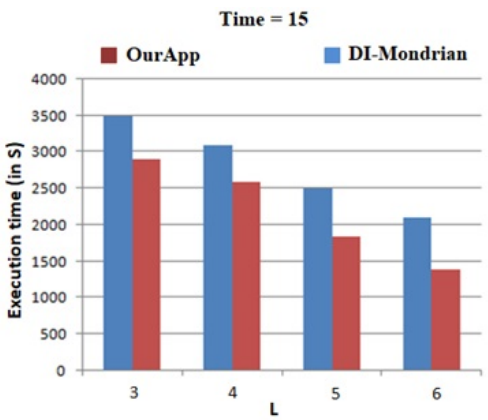

Fig. 8. Execution time for Adult 10M

V. Conclusion

The existing literature lacks techniques for addressing privacy-preserving continuous data publishing in the context of big data. This paper introduces a novel distributed bottom-up anonymization approach using the Apache Spark platform to ensure privacy in continuous big data publishing. The main innovation lies in its capacity to insert diverse records collected from workers into an improved bottom-up R-tree generalization, effectively minimizing information loss. Experimental results show that our approach outperforms Mondrian algorithms in terms of data utility and execution time. In future work, it would be interesting to extend the approach to handle internal updates between publications is a potential direction, along with considering arbitrary updates to enhance the proposed model.

References

[1] P.Jain, M.Gyanchandani, N.Khare, "Privacy and utility preserving data clustering for data anonymization and distribution on Hadoop", Future Generation Computer Systems, Vol.74, pp.393-408, 2017.

[2] L. Sweeney, "k-anonymity: a model for protecting privacy", Int. J. Uncertainty, Fuzziness Knowledge-Based Syst. Vol 10 (05), pp.557–570, 2002.

[3] A. Machanavajjhala, D. Kifer, J. Gehrke, M. Venkitasubramaniam, "l-diversity: Privacy beyond k-anonymity," ACM Trans. Knowl. Discov. from Data, vol 1(1), 2007.

[4] A.Mostefaoui, M.Merzoug, A.Haroun, A.Nassar, Big data architecture for connected vehicles: Feedback and application examples from an automotive group", Future Generation Computer Systems, Vol.134, pp.374-387, 2022

[5] S. Kabou, S. M. Benslimane, and M. Mosteghanemi, "A survey on privacy preserving dynamic data publishing", Int. J. Organizational and Collective Intelligence, vol 8(4), pp.1–20, 2018.

[6] X.Xiao, Y.Tao, "m-invariance: Towards privacy preserving republication of dynamic datasets. In Proc. of ACM International Conference on Management of Data (SIGMOD), Beijing, China. 2007

[7] M. Al-Zobbi, S. Shahrestani, and C. Ruan, "Experimenting sensitivity-based anonymization framework in apache spark", Journal of Big Data, vol 5(1), 2018

[8] B. Fung, K.Wang, R. Chen, P. Yu, "Privacy-preserving data publishing, A survey of recent developments". ACM Computing Surveys, vol 42(4), 2010

[9] M. Zaharia et al, "Apache Spark: a unified engine for big data processing", Communications of the ACM, vol 59 (11), 2016

[10] U. Sopaoglu, O.Abul, "A top-down k-anonymization implementation for apache spark". In Proceedings of the 2017 IEEE International Conference on Big Data, Boston, USA, 11–14 December 2017

[11] H. Zakerzadeh, C.C. Aggarwal, K. Barker, " Privacy-preserving big data publishing", In Proceedings of the 27th International Conference on Scientific and Statistical Database Management, June 2015.

[12] F. Ashkouti, K. Khamforoosh and A. Sheikhahmadi, "DI-Mondrian-Distributed improved Mondrian for satisfaction of the L-diversity privacy model using Apach Spark", Information Sciences, vol. 546, pp. 1-24, 2021

[13] K.R.Pandi Lakshmi, .G.Rasitha Banu, "An advanced bottom up generalization approach for big data on cloud", Int. J. Computer Science and Information Security, vol 12(5), 2014

[14] A. Irudayasamy, L. Arockiam, "Parallel bottom-up generalization approach for data anonymization using map reduce for security of data in public cloud", Indian Journal of Science and Technology, vol. 8, pp. 1, 2015.

[15] S. Kabou and S. M. Benslimane, "A new distributed anonymization protocol with minimal loss of information", Int. J. Organizational and Collective Intelligence, vol 7(1), pp.1–19, 2017.

[16] N.V.S. Lakshmipathi Raju, M.N. Seetaramanath, P. Srinivasa Rao, "An enhanced dynamic KC-slice model for privacy preserving data publishing with multiple sensitive attributes by inducing sensitivity", Journal of King Saud University-CIS, vol 34(01), 1394-1406, 2022.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on June 23,2025 at 11:18:52 UTC from IEEE Xplore. Restrictions apply.